Smart Contract Security Analysis Report

Generated on: 2026-04-21 22:50:20

1. Executive Summary

This report provides a consolidated view of the security analysis performed on 14 smart contract(s). In total, 15 potential vulnerabilities were identified across 10 contract(s).

2. Findings Overview

Contract Name	Total Issues	Status
AccessControl.sol	1	VULNERABLE
DelegatecallUnsafe.sol	1	VULNERABLE
FrontRunning.sol	3	VULNERABLE
Reentrancy.sol	1	VULNERABLE
SelfDestruct.sol	1	VULNERABLE
TimestampDependence.sol	4	VULNERABLE
TxOrigin.sol	1	VULNERABLE
UncheckedCalls.sol	0	SECURE
contract_1.sol	1	VULNERABLE
contract_2.sol	0	SECURE
contract_3.sol	0	SECURE
contract_4.sol	1	VULNERABLE
contract_5.sol	0	SECURE
contract_7.sol	1	VULNERABLE

3. Detailed Vulnerability Analyses

Smart Contract Security Analysis Report

Generated on: 2026-04-21 22:50:20

Contract: AccessControl.sol

Static Analysis Findings (Slither)

[CRITICAL] Missing Access Control

Function: changeOwner | Line: 13

Explanation:

The function 'changeOwner' updates the sensitive variable 'owner' but does not seem to have any access control (like onlyOwner). This allows any external actor to take control of the contract.

Suggest

Add an access control modifier (e.g., 'onlyOwner') to the function 'changeOwner' at line 13.

Contract: contract_1.sol

Static Analysis Findings (Slither)

[HIGH] Reentrancy (CEI Violation)

Function: withdraw | Line: 13

Explanation:

The external call at line 13 occurs before state variables (balances) are updated at line 14. This violates the Checks-Effects-Interactions pattern.

Suggest

Reorder the operations in function 'withdraw' to update the state variables (balances) before the external call at line 13.

Contract: contract_4.sol

Static Analysis Findings (Slither)

[HIGH] Reentrancy (CEI Violation)

Function: withdraw | Line: 11

Explanation:

The external call at line 11 occurs before state variables (balances) are updated at line 13. This violates the Checks-Effects-Interactions pattern.

Suggest

Reorder the operations in function 'withdraw' to update the state variables (balances) before the external call at line 11.

Contract: contract_7.sol

Static Analysis Findings (Slither)

[HIGH] Reentrancy (CEI Violation)

Function: withdraw | Line: 10

Explanation:

The external call at line 10 occurs before state variables (balances) are updated at line 13. This violates the Checks-Effects-Interactions pattern.

Suggest

Reorder the operations in function 'withdraw' to update the state variables (balances) before the external call at line 10.

Contract: DelegatecallUnsafe.sol

Static Analysis Findings (Slither)

[CRITICAL] Dangerous Delegatecall

Function: executeWith | Line: 38

Explanation:

Function 'executeWith' performs a delegatecall to a non-constant address ('_target') at line 38. delegatecall executes foreign bytecode inside this contract's own storage context. If the destination is user-controlled or can be changed by an attacker, they can destroy storage, steal ownership, or drain funds.

Suggest

Ensure the delegatecall target is a hardcoded constant or an immutable variable set only in the constructor. Never delegatecall to an address supplied by msg.sender or stored in a mutable state variable. Consider using OpenZeppelin's upgradeable proxy patterns which enforce these constraints.

Contract: FrontRunning.sol

Static Analysis Findings (Slither)

[MEDIUM] Front-Running: ERC-20 Approve Race

Function: approve | Line: 22

Explanation:

The 'approve' function sets a new allowance directly. If an approved spender is watching the mempool, they can call transferFrom() before the allowance update is mined, then again after ? effectively spending both the old and new allowance.

Suggest

Replace direct approve() with increaseAllowance() / decreaseAllowance() (OpenZeppelin pattern). Alternatively, require the sender to set allowance to 0 before changing it: require(currentAllowance == 0 || newAmount == 0).

[MEDIUM] Front-Running: Timestamp-Dependent Payable

Function: flashSale | Line: 49

Explanation:

Payable function 'flashSale' uses block.timestamp to determine outcome (e.g., auction deadlines, lottery windows). An attacker or miner can observe this transaction and front-run it, or manipulate the timestamp by ~15s to change which transaction wins.

Suggest

Smart Contract Security Analysis Report

Generated on: 2026-04-21 22:50:20

Use a commit-reveal scheme to hide transaction intent until after the block is committed. For auctions, consider a blind auction pattern. Avoid using block.timestamp as the sole decider for financial outcomes.

[MEDIUM] Timestamp Dependence

Function: flashSale | Line: 50

Explanation:

Function 'flashSale' uses 'block.timestamp' in conditional logic at line 50. Miners can manipulate this value by up to ~15 seconds, potentially influencing time-sensitive operations such as timelocks, auctions, or random-seed generation.

Suggest

Avoid using block.timestamp for critical randomness or precise timing. Use a commit-reveal scheme for randomness, or design timelocks with sufficiently large windows (e.g., > 15 minutes) so miner manipulation is economically insignificant.

Contract: Reentrancy.sol

Static Analysis Findings (Slither)

[HIGH] Reentrancy (CEI Violation)

Function: withdraw | Line: 16

Explanation:

The external call at line 16 occurs before state variables (balances) are updated at line 19. This violates the Checks-Effects-Interactions pattern.

Suggest

Reorder the operations in function 'withdraw' to update the state variables (balances) before the external call at line 16.

Contract: SelfDestruct.sol

Static Analysis Findings (Slither)

[CRITICAL] Unprotected Self-Destruct

Function: kill | Line: 13

Explanation:

The function 'kill' contains a self-destruct operation at line 13 and lacks proper access control. Any external user can call this to destroy the contract.

Suggest

Restrict access to the function 'kill' using an 'onlyOwner' modifier or remove the self-destruct call.

Contract: TimestampDependence.sol

Static Analysis Findings (Slither)

[MEDIUM] Front-Running: Timestamp-Dependent Payable

Function: bid | Line: 18

Explanation:

Payable function 'bid' uses block.timestamp to determine outcome (e.g., auction deadlines, lottery windows). An attacker or miner can observe this transaction and front-run it, or manipulate the timestamp by ~15s to change which transaction wins.

Suggest

Use a commit-reveal scheme to hide transaction intent until after the block is committed. For auctions, consider a blind auction pattern. Avoid using block.timestamp as the sole decider for financial outcomes.

[HIGH] Reentrancy (CEI Violation)

Function: bid | Line: 24

Explanation:

The external call at line 24 occurs before state variables (highestBidder) are updated at line 26. This violates the Checks-Effects-Interactions pattern.

Suggest

Reorder the operations in function 'bid' to update the state variables (highestBidder) before the external call at line 24.

[MEDIUM] Timestamp Dependence

Function: bid | Line: 20

Explanation:

Function 'bid' uses 'block.timestamp' in conditional logic at line 20. Miners can manipulate this value by up to ~15 seconds, potentially influencing time-sensitive operations such as timelocks, auctions, or random-seed generation.

Suggest

Avoid using block.timestamp for critical randomness or precise timing. Use a commit-reveal scheme for randomness, or design timelocks with sufficiently large windows (e.g., > 15 minutes) so miner manipulation is economically insignificant.

[MEDIUM] Timestamp Dependence

Function: claimPrize | Line: 32

Explanation:

Function 'claimPrize' uses 'block.timestamp' in conditional logic at line 32. Miners can manipulate this value by up to ~15 seconds, potentially influencing time-sensitive operations such as timelocks, auctions, or random-seed generation.

Suggest

Avoid using block.timestamp for critical randomness or precise timing. Use a commit-reveal scheme for randomness, or design timelocks with sufficiently large windows (e.g., > 15 minutes) so miner manipulation is economically insignificant.

Contract: TxOrigin.sol

Static Analysis Findings (Slither)

[HIGH] Vulnerable use of tx.origin

Function: withdrawAll | Line: 13

Explanation:

The function 'withdrawAll' uses 'tx.origin' for authorization at line 13. This makes the contract vulnerable to phishing attacks where an attacker can trick a user into calling a malicious contract that then calls this function.

Suggest

Replace 'tx.origin' with 'msg.sender' in function 'withdrawAll' to ensure only the immediate caller is authenticated.